

Companion di studio — Modulo 1 (materiale extra per l'orale)

Materiale **aggiuntivo** per preparare l'orale del Modulo 1 (vectorization del partition mapping kernel). Non modifica né il report né il codice consegnati: sta tutto in `module_1/extra_experiments/`. È complementare al PDF `analisi-report-M1-SPM.pdf` (che fa il walkthrough sezione per sezione): qui trovi le **risposte ai dubbi aperti** e i **cinque esperimenti nuovi**, tutti misurati su **node09** (AMD EPYC 7301, lo stesso nodo del report), con grafici e numeri riproducibili.

Regola seguita ovunque: nessun numero inventato. Ciò che non è misurato è dichiarato tale.

1. Risposte rapide ai dubbi del todo

“MKeys sta per?” Mkeys/s = milioni di chiavi al secondo ($M = 10^6$). È la metrica di throughput della Tabella 1. Es. baseline ~ 910 Mkeys/s = $9.1 \cdot 10^8$ chiavi/s.

“Fig. 2, cos'è key space size?” Nel report compilato la Figura 2 è quella combinata (P-sweep + key-space sweep); il pannello (b) ha “key-space size” sull'asse x. È il parametro `key_space`: la **cardinalità dell'universo delle chiavi**. Nel generatore (`common.hpp`), se `key_space > 0` allora `keys[i] = rng() % key_space`; piccolo $\rightarrow$ poche chiavi distinte $\rightarrow$ tanti duplicati. Il grafico mostra che il **throughput** non dipende da `key_space` (kernel branchless, costo costante per chiave). Attenzione: `key_space` non tocca il throughput ma **sì** il bilanciamento (vedi esperimento 1: a `key_space=1000` lo sbilanciamento sale a 1.28).

“È stata fatta una run che mette insieme AVX2 + autovec, o solo il confronto?” Solo il confronto, ed è corretto così. I tre binari sono strategie **distinte** sulla stessa hash: baseline (`-fno-tree-vectorize`), autovec (vettorizzato dal compilatore), `avx2` (intrinsic a mano). Non ha senso “fonderle”: un loop o è intrinsic a mano o è auto-vettorizzato. Nota tecnica: l'`avx2` è comunque compilato con `-O3` (quindi l'autovec è attiva), ma agisce solo su coda scalare e checksum, non sul loop intrinsic.

“Data layout e allineamento a 32 B?” Chiavi (`uint64`) e partition id (`uint32`) sono in **array separati contigui, stride unitario** (Structure-of-Arrays). In un kernel bandwidth-bound questo significa che ogni cache line caricata contiene solo byte utili e gli accessi sono sequenziali: è la condizione per load/store vettoriali. L'allineamento a **32 byte** (`posix_memalign`) serve alle load AVX2 allineate (`vmovdqa / _mm256_load`): evita la penalità di *split-line* quando un accesso a 256 bit attraversa il confine di due cache line. P potenza di due rende il modulo uno shift ($>> (32 - \log_2 P)$).

Gli altri punti (32 vs 64 bit, flags, 96% della banda, rerun CUDA) hanno un esperimento dedicato: vedi sotto.

2. Walkthrough del report con gli agganci agli esperimenti

sezione del report	cosa dice	come difenderla / esperimento
1.1 Mapping function	Fibonacci multiply-shift 32-bit, XOR-fold, costante aurea	Esp. 1: fib32 è robusta dove mod collassa; ogni pezzo (fold, shift, 32 bit) ha un caso avversario
1.2 Data layout	SoA contiguo, allineato 32 B, P pot. di 2	vedi §1: byte utili per cache line, no split-line, modulo = shift
2. Auto-vectorization	GCC vettorizza il loop (32 B ymm + 16 B xmm), vec-missed vuoto	Esp. 3: il salto O2 $\rightarrow$ O3 (+33%) è la vettorizzazione; baseline vs autovec = 1.46×
3. AVX2 intrinsics	8 chiavi/iter, permute + <code>vpmulld</code> , coda scalare	Esp. 4: <code>vpmulld</code> nativo (1 istr, 8 chiavi) è il motivo dei 32 bit
4.2 Why speedup limited	$I=0.33$, memory-bound, 96% del tetto single-core	Esp. 2: il tetto matched misurato è 19.15 GB/s $\rightarrow$ autovec $\sim 83\%$, non 96%

sezione del report	cosa dice	come difenderla / esperimento
4.3 Autovec vs AVX2	l'autovec batte le intrinsics (banda meglio saturata)	Esp. 3/5: autovec 1330 > avx2 1199, confermato su node09
5. CUDA	kernel 808 GB/s, PCIe = 98%, e2e 1.12x	Esp. 5: riprodotto (kernel ~800 GB/s), ma e2e dipende dall'affinità NUMA

3. I cinque esperimenti (sintesi, grafico, tesi)

Esp. 1 — Qualità della hash (01_hash_quality/)

Attenzione: la hash del progetto è **fib32** (32 bit). **fib64** è solo un termine di paragone che ho aggiunto, non è quella usata.

La hash del report ha 3 ingredienti: (1) XOR-fold dei 32 bit alti nei bassi, (2) moltiplicazione per la costante aurea, (3) prende i bit ALTI del prodotto. Le 5 funzioni confrontate isolano, una alla volta, perché serve ogni ingrediente:

- **fib32 (report):** la ricetta completa. Riferimento.
- **fib64:** stessa idea a 64 bit, mostra che 32 vs 64 bit danno qualità identica.
- **mod (naive):** $k \% P$, solo i bit bassi della chiave, niente moltiplicazione.
- **fib32 low-bits:** come fib32 ma prende i bit BASSI del prodotto, isola “perché i bit alti”.
- **mult32 no-fold:** come fib32 ma senza XOR-fold, isola “perché il fold”.

Le 7 colonne sono tipi di chiavi in input: uniform (random), sequential ($k=i$, row-id), strided (multipli di 4096, offset allineati), low16=0 e high32-only (entropia solo nei bit alti), e le due dup, che hanno un universo di soli 1000 valori distinti ($N=10^8$ chiavi, quindi $\sim 10^5$ duplicati per valore): dup contigui sono i valori 0...999, dup strutturati sono gli stessi valori moltiplicati per 65536. Le due dup sono la stessa distribuzione (stesso seme, stessa sequenza, stesse molteplicità): cambia solo la codifica in bit.

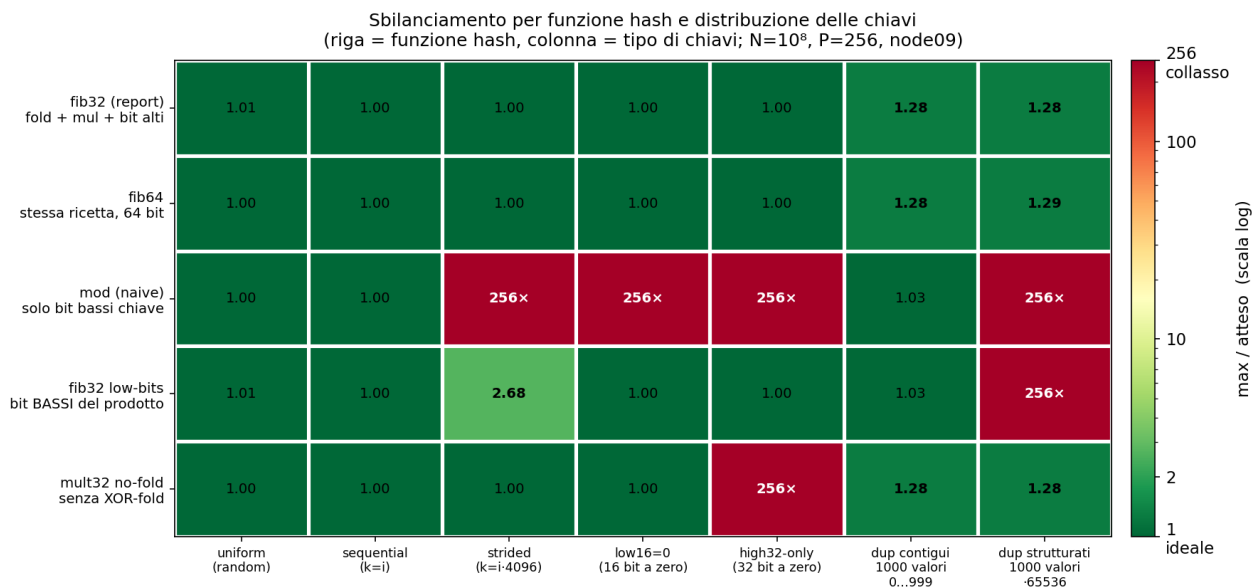


Figure 1: Matrice hash x distribuzione delle chiavi.

Risultato: su chiavi **uniformi vanno bene tutte** (fib32 = 1.01). Su chiavi **strutturate** mod **collassa** (256x: tutte le 10^8 chiavi in 1 partizione) ogni volta che i bit bassi mancano di entropia; fib32 low-bits sbaglia su strided; mult32 no-fold sbaglia su high32-only. Solo **fib32 e fib64 reggono ovunque**, con qualità identica.

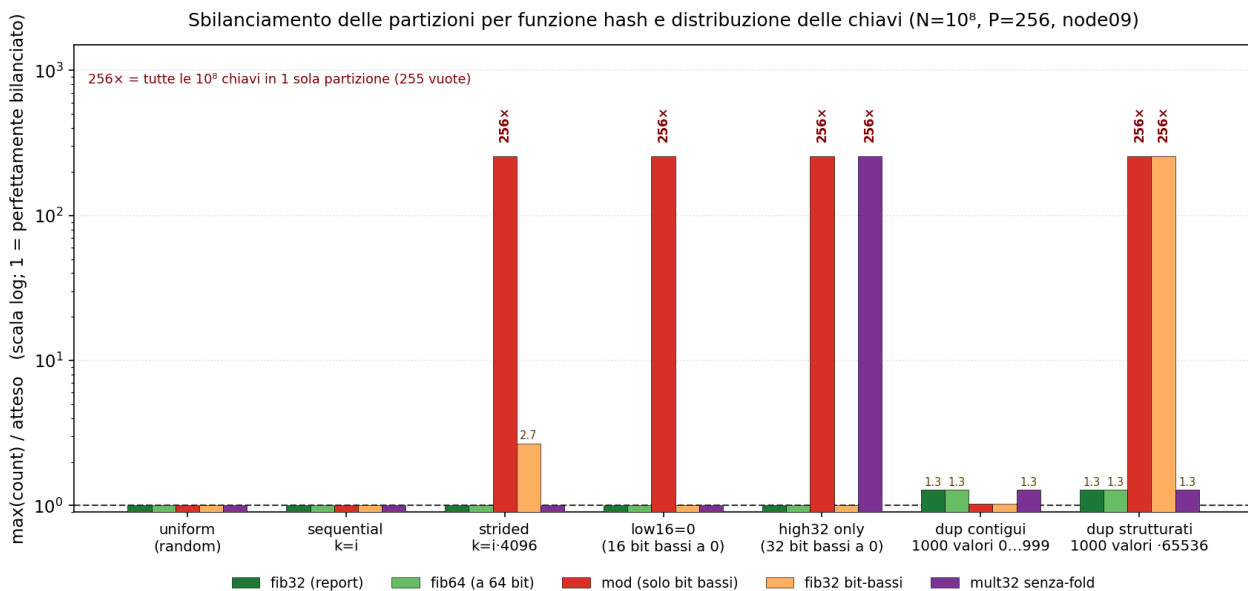
Sul caso dup (dove mod sembra vincere): nelle ultime due colonne della matrice, mod fa 1.03 su dup contigui (valori 0...999) ma **256x su dup strutturati** (stessi 1000 valori distinti, stesse molteplicità, moltiplicati per 65536). fib32 resta 1.28 in entrambi. Il vantaggio di mod esiste solo perché i valori erano contigui: basta riscalarli di 2^{16} e i $\log_2 P$ bit bassi vanno a zero, quindi collassa. **Il default del progetto è comunque uniform (key_space=0), non dup.**

Le due cifre e il loro meccanismo:

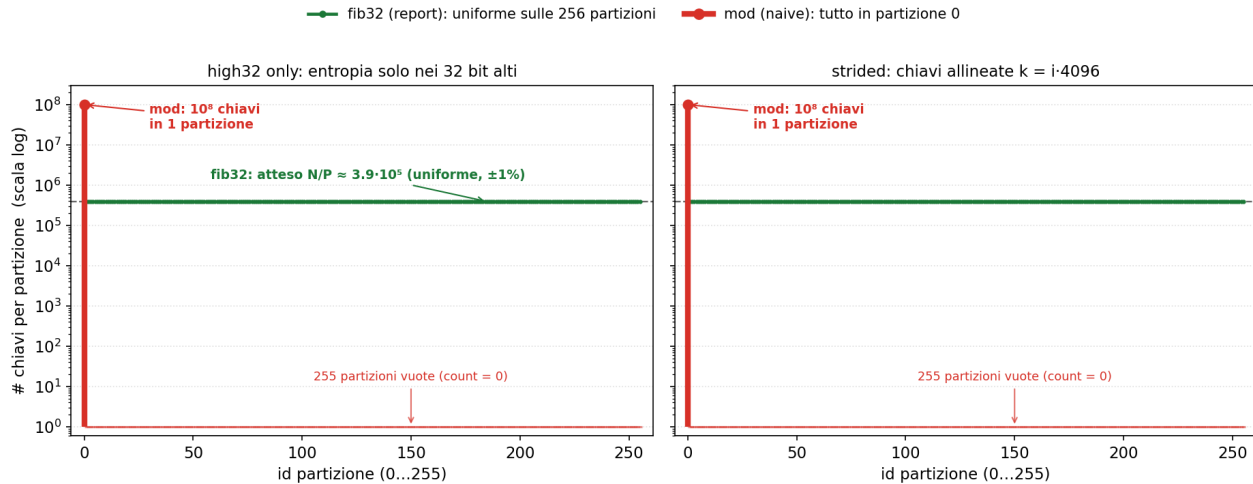
- **Perché mod fa esattamente 1.03 sui dup contigui.** $k \& 255$ su 0...999 è un round-robin: $1000 = 3 \cdot 256 + 232$, quindi 232 partizioni prendono 4 valori e 24 ne prendono 3 $\rightarrow 4/3.906 = 1.024$ (misurato 1.029, il resto è fluttuazione dei conteggi).
- **Perché fib32 fa 1.28 e non 1.00.** Non è un difetto della hash: i duplicati di un valore vanno tutti nella stessa partizione, quindi con 1000 valori distinti su $P=256$ la media è 3.906 valori per partizione e **nessuna hash può scendere sotto $4/3.906 = 1.024$** . È il limite strutturale di $\text{key_space}/P$. fib32 sparpaglia senza sapere che i valori sono contigui e arriva a 5 valori nella partizione più carica $\rightarrow 5/3.906 = 1.28$.

Tesi (due decisioni separate): (1) fib invece di mod per robustezza: mod collassa su 4 distribuzioni su 7, fib32 mai, e il suo worst case 1.28 è quantizzazione strutturale. In un hash join la distribuzione delle chiavi non è nota a priori, quindi la scelta cade sulla garanzia indipendente dai dati. (2) 32 bit invece di 64 **non** si decide qui, perché fib32 e fib64 hanno qualità identica: si decide su AVX2 (Esp. 4).

Le stesse cifre in barre (magnitudine dei collassi) e lo zoom sull'occupazione delle 256 partizioni in un caso di collasso:



Occupazione delle 256 partizioni su chiavi strutturate: fib32 bilancia, mod collassa



Esp. 2 — Tetto di banda / il “96%” (02_bandwidth_ceiling/)

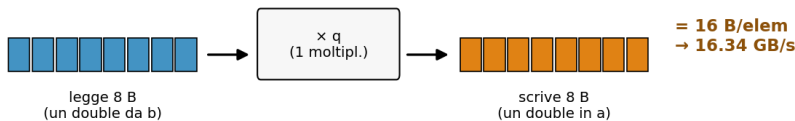
Il 16.4 GB/s del report è dichiarato “misurato” ma **non c’è alcun artefatto** che lo produca. Misurato su node09: STREAM Copy 26.6, Triad 19.3, **Scale 16.34** ($\approx$ il 16.4 del report), e una **copia matched** al pattern del kernel (read u64 + write u32) = **19.15 GB/s**. L’autovec (15.9 GB/s) è al **83%** del tetto matched, **non 96%**. Il kernel resta memory-bound, ma il “96%” era ottimistico (denominatore = STREAM Scale invece del Copy). **Tesi:** regime memory-bound corretto; percentuale di saturazione da correggere a ~83%.

Perché il tetto giusto è ~19 e non 16.4: dipende dal **pattern di byte**. Il 16.4 è lo STREAM Scale ($a[i]=q \cdot b[i]$ su double: legge 8 B, scrive 8 B); il kernel legge un uint64 (8 B) e scrive un uint32 (**4 B**), pattern più leggero che sostiene ~19 GB/s.

byte letti byte scritti byte non scritti (risparmiati dal kernel)

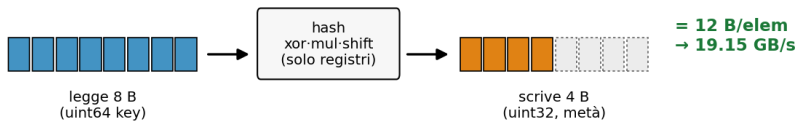
STREAM Scale: il tetto generico, 16.4 GB/s

loop: $a[i] = q \cdot b[i]$ (array di double, 8 byte)



Kernel partition map: il tetto matched, 19.15 GB/s

loop: $part_ids[i] = hash(keys[i])$ (uint64 in, uint32 out)



Stessa lettura (8 B), ma il kernel scrive solo 4 B invece di 8 → pattern più leggero → tetto giusto 19, non 16. Confrontare col 16.4 gonfia la saturazione (96% invece di 83%).

Figure 2: Pattern di byte: STREAM Scale (16.4) legge 8 e scrive 8, il kernel legge 8 e scrive 4 (tetto matched 19.15).

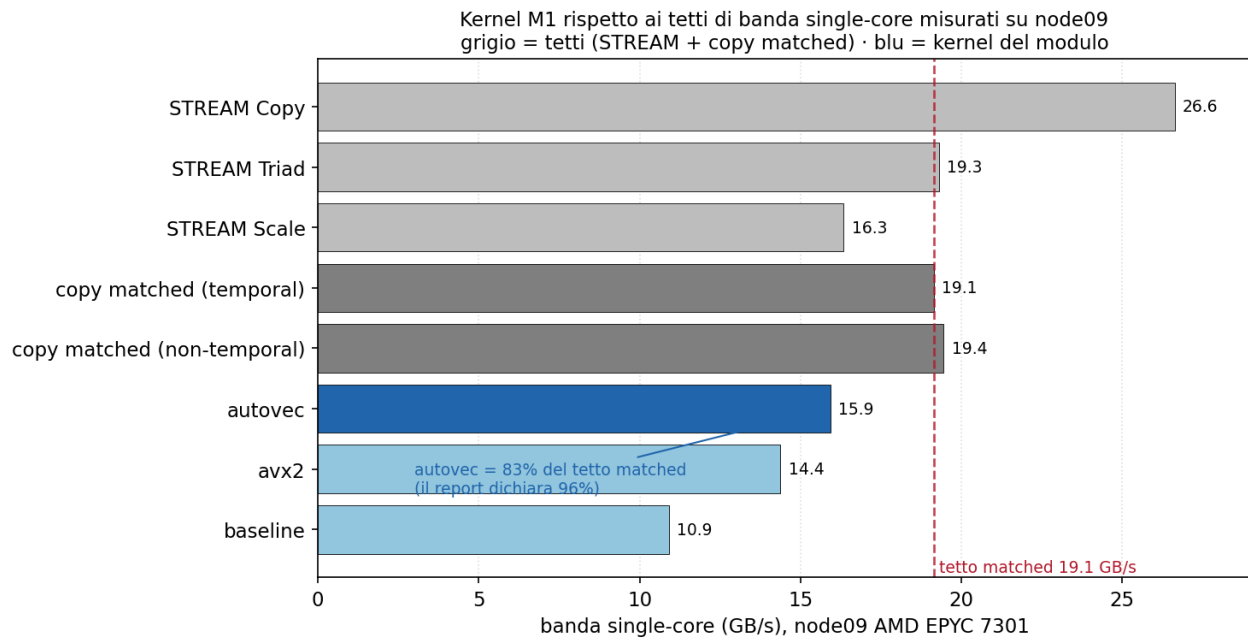


Figure 3: Kernel vs tetti di banda single-core misurati su node09.

Esp. 3 — Grid search dei flag (03_flags_gridsearch/)

Il report usa per l'autovec un set di flag fisso senza giustificarlo. La grid (a scala, da O0 naive 145 Mkeys/s all'autovec 1325) e soprattutto l'**ablation** (parti dalla config consegnata e togli un flag alla volta) mostrano il contributo di ciascuno.

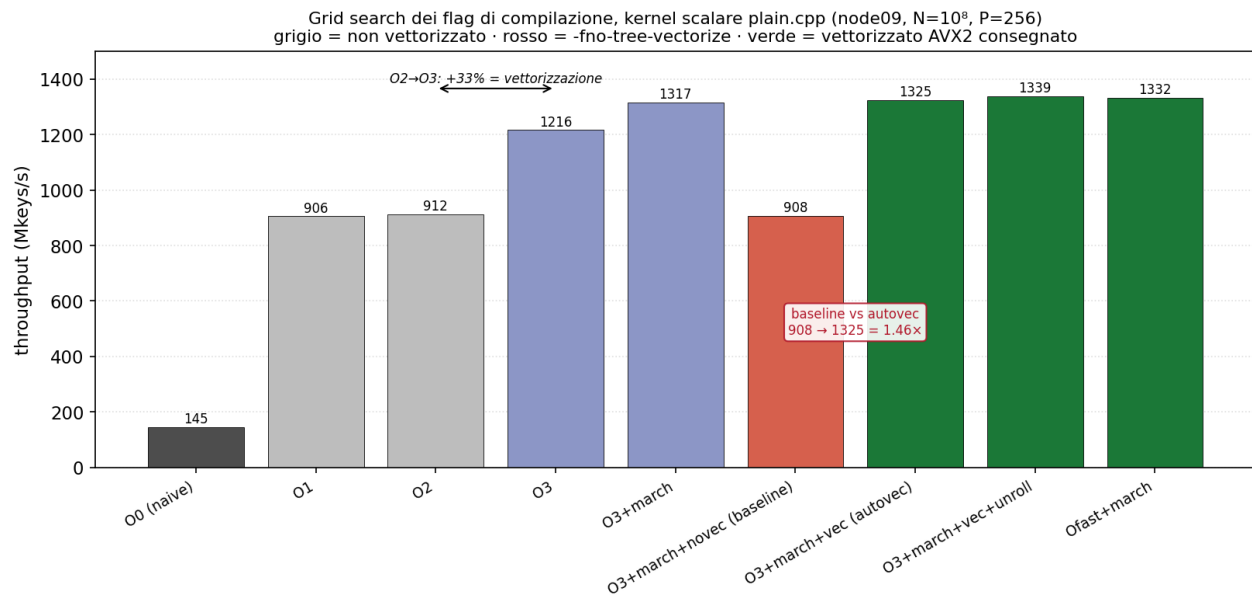


Figure 4: Grid search dei flag: throughput per configurazione (O0 -> Ofast).

Ablation dalla config autovec (la vista che giustifica esattamente i flag scelti): togliere `-ftree-vectorize` costa **-31%**, togliere `-march=native` **-15%**; invece O3->O2->O1, `+funroll-loops` e `-Ofast` sono **neutri** (~0%). Cioè lo speedup poggia solo su **due flag** (vettorizzazione + march); il livello di ottimizzazione, una volta accesa la vettorizzazione, non conta. **Tesi:** i flag del Makefile sono la scelta minimale corretta, niente di superfluo.

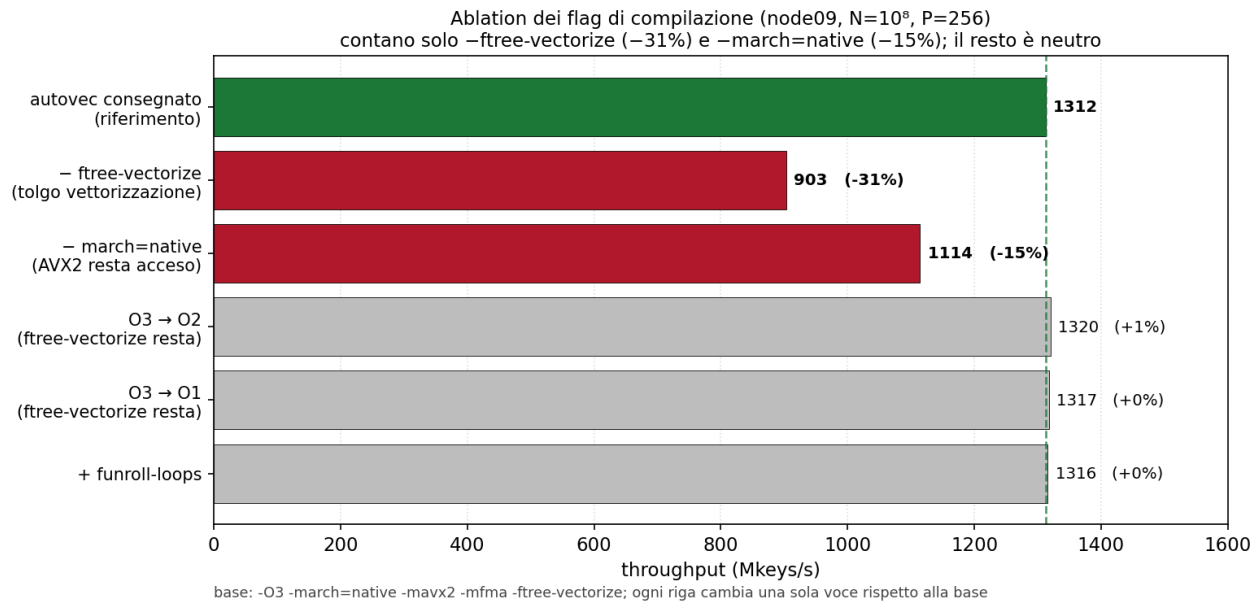


Figure 5: Ablation dei flag dalla config autovec: contano solo vettorizzazione e march.

Cosa fa ogni flag (dettaglio)

- **-O0...-O3**: livelli crescenti. `-O0` è la traduzione letterale (variabili in RAM, niente inlining): è il naive a 145 Mkeys/s. `-O1` accende la register allocation ed è il salto più grande (906). `-O2` aggiunge scheduling e inlining ma **non** l'auto-vettorizzazione dei loop. `-O3` accende `-ftree-vectorize` di default: il +33% su O2 è la vettorizzazione.
- **-Ofast** = `-O3 + -ffast-math` (rilassa lo standard IEEE sui float). Kernel a interi -> fast-math inutile, identico a O3, e rischioso su codice FP. Escluso a ragione.
- **-march=native**: genera codice per la CPU esatta (EPYC 7301, Zen1). Due effetti: (1) abilita i set di istruzioni (AVX2, FMA): senza, la vettorizzazione usa SSE a 128 bit (4×uint32); con, AVX2 a 256 bit (8×uint32); (2) tuning per la microarchitettura. Vale da +8% (vs O3 generico) a +15% (ablation). Nota: Zen1 ha unità SIMD fisicamente a 128 bit, quindi una AVX2 a 256 bit è spezzata in due micro-op -> su questa CPU conta più il tuning corretto che la larghezza.
- **-mavx2**: abilita esplicitamente AVX2. Con `-march=native` è già implicato -> in parte ridondante, è lì per chiarezza.
- **-mfma**: abilita le fused multiply-add (float). Il kernel è a interi -> **non usata**: innocua ma inutile qui.
- **-ftree-vectorize / -fno-tree-vectorize**: il flag centrale. Accende/spegne l'auto-vettorizzatore (loop scalare 1 chiave/iter -> loop SIMD 8 chiavi/iter). Leva più forte: -31% se tolto. La baseline è "autovec con questo OFF".
- **-funroll-loops**: replica il corpo del loop per ridurre l'overhead di conteggio/branch. In un kernel memory-bound l'overhead è trascurabile -> +1%. Non nel Makefile.
- **-fopt-info-vec-optimized/-missed**: non cambiano il codice, fanno **stampare** quali loop GCC ha vettorizzato (optimized) o tentato invano (missed, vuoto qui). Servono a provare che la vettorizzazione è avvenuta.

Perché il loop è vettorizzabile: trip count noto, iterazioni indipendenti, nessuna chiamata nel corpo, `__restrict__` (keys e part_ids non si sovrappongono -> il compilatore può riordinare), stride unitario, nessun branch, e l'operazione centrale è una mul a 32 bit con istruzione nativa (`vpmulld`).

Esp. 4 — Counterfactual 32 vs 64 bit (`04_hash64_counterfactual/`)

AVX2 a 64 bit implementato con la decomposizione a **3× vpmuludq** (confermata nell'objdump). Misura: `avx2_64/scalar64` = 0.98× (la SIMD peggiora), contro `avx2_32/scalar32` = 1.32×. Sfumatura: `scalar64` (1100) > `scalar32` (910) perché la hash a 64 bit è più semplice in scalare; i 32 bit si scelgono perché **vettorizzano**, non perché più economici. **Tesi**: claim del report confermato con numeri.

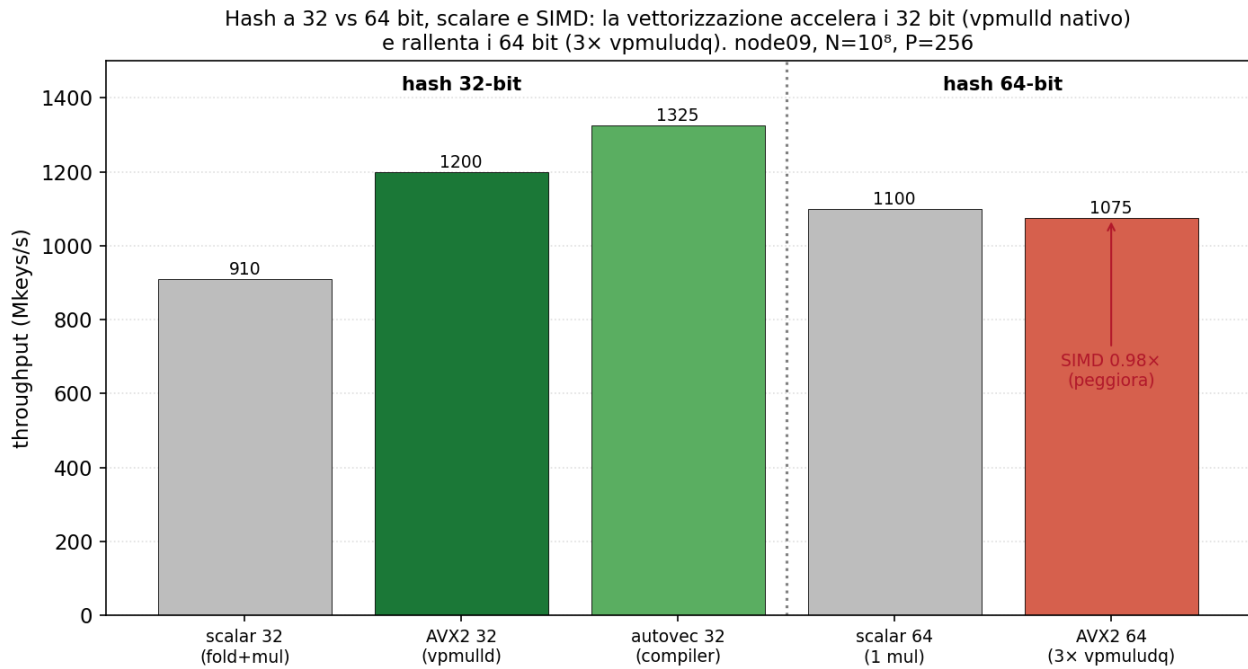


Figure 6: Hash 32 vs 64 bit: la SIMD aiuta i 32 bit (vpmulld), peggiora i 64 bit (3x vpmuludq).

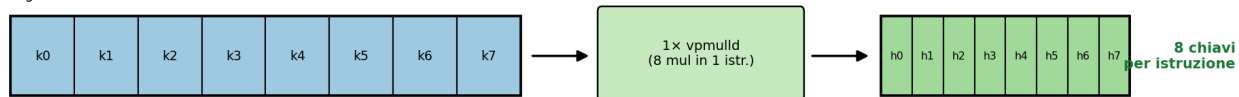
Perché la SIMD a 64 bit perde (dettaglio) Il nocciolo è uno solo: **AVX2 ha la mul intera nativa a 32 bit** (vpmulld, 8 mul in una istruzione) **ma NON a 64 bit** (vpmullq arriva solo con AVX-512). L'unico mattone a 64 bit è vpmuludq (32x32->64). Per una mul 64x64 servono più vpmuludq "in colonna": con $k = 2^{32} \cdot k_{hi} + k_{lo}$ e $A = 2^{32} \cdot A_{hi} + A_{lo}$, i 64 bit bassi del prodotto sono

$$\text{low64} = k_{lo} \cdot A_{lo} + ((k_{hi} \cdot A_{lo} + k_{lo} \cdot A_{hi}) \ll 32)$$

cioè tre prodotti 32x32 = **3 vpmuludq** + shift + add, contro **1 vpmulld** del caso 32 bit (confermato nell'objdump: 3 vpmuludq nel loop a 64 bit). Doppio svantaggio, visibile nel diagramma: (1) ~7 istruzioni per la mul invece di 1; (2) metà chiavi per registro, perché 256/64 = 4 corsie a 64 bit contro 256/32 = 8 corsie a 32 bit.

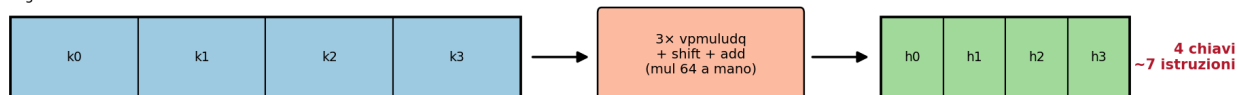
Hash 32 bit: vpmulld (istruzione nativa)

registro AVX2 da 256 bit = 8 corsie a 32 bit



Hash 64 bit: vpmullq non esiste in AVX2

registro AVX2 da 256 bit = solo 4 corsie a 64 bit



Stesso registro da 256 bit: a 64 bit metà chiavi (4 invece di 8) e ~7x lavoro per mul → la SIMD perde: $avx2_64 = 0.98\times$ (peggiora), $avx2_32 = 1.32\times$ (aiuta).

Figure 7: Registro AVX2 da 256 bit: 8 chiavi con 1 `vpmulld` (32 bit) vs 4 chiavi con 3 `vpmulldq` (64 bit).

Risultato misurato: $avx2_64/scalar64 = 0.98\times$ (la SIMD **peggiora**) contro $avx2_32/scalar32 = 1.32\times$. In regime memory-bound lo scalare ha già il calcolo nascosto sotto la latenza di memoria; la decomposizione vettoriale aggiunge solo pressione di istruzioni → finisce sotto lo scalare.

Sorpresa da citare: scalar64 (1100) > scalar32 (910). In scalare la hash 64 bit è UNA mul nativa; la 32 bit fa di più (estrai `k_lo`, estrai `k_hi` con shift, XOR, poi mul). Quindi i 32 bit **non** si scelgono perché più economici in scalare (non lo sono), ma solo perché **vettorizzano**; e la qualità è identica ($fib32 = fib64$, esp. 1). Albero decisionale: 64 bit = più semplice in scalare, qualità uguale, ma non vettorizza; 32 bit = un filo più di lavoro scalare, qualità uguale, ma vettorizza nativo → poiché il compito è vettorizzare, vince 32 bit. Classifica: $autovec32 (1325) > avx2_32 (1199) > scalar64 (1100) > avx2_64 (1075) > scalar32 (910)$.

Esp. 5 — Rerun CPU + CUDA (05_rerun/)

Tabella 1 CPU riprodotta (stessi speedup, stessi checksum). CUDA: kernel ~800 GB/s riprodotto, ma l'**end-to-end dipende dall'affinità NUMA**: bindando al dominio della GPU (node 4) → e2e 1000 Mkeys/s (≈ il 1031 del report); sul socket lontano (node 0) → 574 (la GPU perde contro la CPU). **Tesi**: il numero del report è riproducibile ma fragile al placement; la conclusione "GPU non conviene per il kernel isolato" regge in ogni caso.

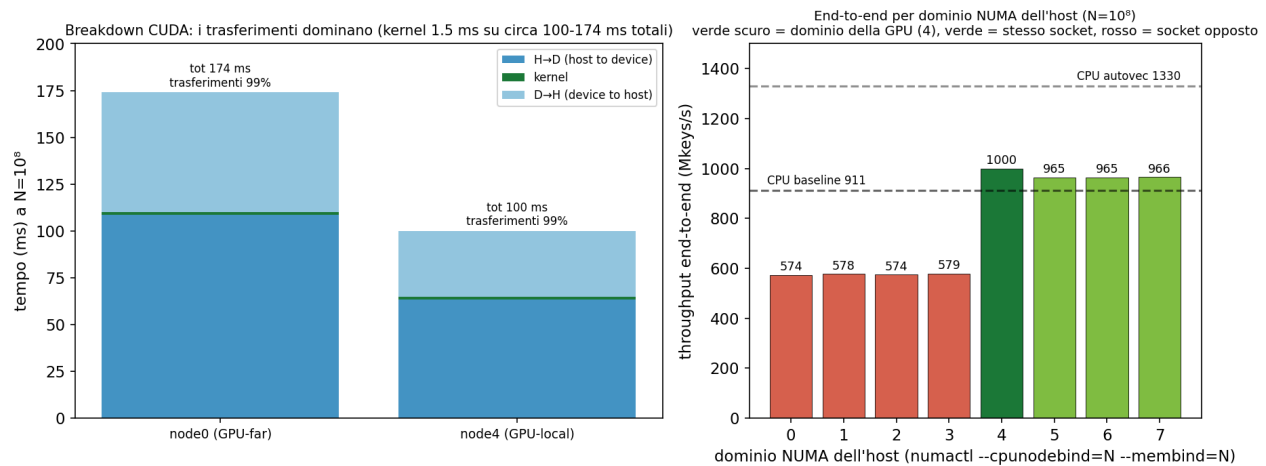


Figure 8: Breakdown CUDA e sensibilit  NUMA: GPU-far perde, GPU-local pareggia la CPU.

4. Onest : i due punti dove il report va oltre il misurato

Da dire spontaneamente all'orale (mostra spirito critico, e sono cose che il prof potrebbe chiedere):

1. Il **"96% del tetto single-core"** poggia su un tetto (16.4 GB/s) non ancorato da alcuna misura nel repo. Misurando su node09, il tetto corretto per questo pattern   ~19.15 GB/s e l'autovec   al ~83%. Il regime memory-bound resta valido;   la percentuale a essere ottimistica (Esp. 2).
2. Il **CUDA end-to-end 1031 Mkeys/s** assume implicitamente che l'host giri sul dominio NUMA della GPU. Senza binding, su node09 pu  scendere a 574 (banda PCIe dimezzata). Il codice consegnato non pinna, quindi il valore dipende dallo scheduler (Esp. 5).

Entrambi **rafforzano** la tesi del modulo invece di indebolirla: il kernel   memory-bound e la GPU non conviene; i due numeri vanno solo riportati con pi  precisione.

5. Cheat-sheet orale M1 (numeri da ricordare)

- Hash: $h(k) = ((k_{lo} \text{ XOR } k_{hi}) \cdot 0x9E3779B9) \gg (32 - \log_2 P)$, $A_{32} = \text{floor}(2^{32}/\phi)$.
- Intensit  operativa $I = 4 \text{ op} / 12 \text{ B} \approx 0.33$ -> memory-bound (traffico reale con RFO ~16 B/chave).
- CPU node09 (N=10⁸, P=256): baseline ~910, autovec ~1330 (1.46 ), avx2 ~1199 (1.31 ) Mkeys/s.
- Tetto matched single-core misurato: **19.15 GB/s** -> autovec al ~83% (non 96%).
- 32 vs 64 bit: $\text{avx2_32}/\text{scalar32} = 1.32\times$, $\text{avx2_64}/\text{scalar64} = 0.98\times$ (3  vpmuludq).
- CUDA: kernel ~800 GB/s (81% HBM2 A30); e2e 1000 (GPU-local) / 574 (GPU-far); PCIe = 98% del tempo.
- Distribuzione fib32 su chiavi uniformi: max/atteso = 1.005; mod collassa a 256  su chiavi strutturate.

Ogni cartella NN_*/ ha il suo README.md con metodo, comando esatto (s run ...), tabella completa e "come difenderlo".